Linux Shell Scripting



Complete Guide for DevOps, Automation & Interview Preparation

What's Inside ?

1. Introduction to Shell Scripting
2. Linux Shell & Basics
3. Variables & User Input
4. Operators
5. Conditional Statements
6. Case Statements
7. Loops (for, while, until)
8. Functions
9. Arrays & Strings
10. Command Line Arguments
11. File & Directory Operations
12. Text Processing (grep, awk, sed)
13. Exit Status & Error Handling
14. Scheduling with Cron
15. System Monitoring Scripts
16. Log Management & Backup
17. DevOps Real-World Scripts
18. Advanced Shell Scripting
19. Best Practices
20. Interview Questions & Practical Tasks


```
[rakesh@linux ~]$ bash script.sh
Hello, DevOps!
Automate with Shell Scripting
$ _
```

Automation | Task Scheduling | System Administration

Monitoring | Log Management | DevOps Support

Learn → Practice → Automate

Why Learn Shell Scripting?

- ✓ Automate daily DevOps tasks
- ✓ Save time & reduce manual work
- ✓ Useful for interviews
- ✓ Essential for production support
- ✓ Works on all Linux systems

> Small Scripts
> Big Automation
> DevOps Power!

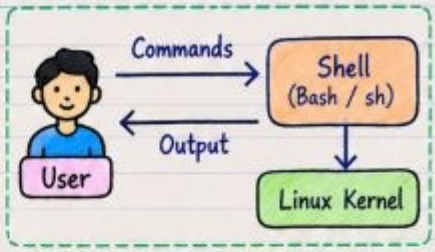
Write
Script
Automate
Save Time

Manjunatha HG

1. Introduction to Shell Scripting

1.1 What is a Shell ?

- A shell is a command line interface between the user and the Linux operating system.
- It takes your commands, interprets them and passes them to the kernel for execution.
- Shell provides a way to interact with the system using commands or scripts.



1.2 What is Shell Scripting ?

- Shell scripting is a way to write a series of Linux commands in a file and execute them together.
- A shell script is a plain text file containing commands, logic and automation tasks.
- It is mostly written in **Bash** (Bourne Again Shell).
- Helps to automate repetitive tasks, system administration, DevOps work and more.

1.3 Types of Shells

- Linux supports different types of shells.
- Most commonly used shell is Bash.
- Bash is default shell in most Linux distributions.

1.4 First Shell Script Example

- Create a simple script to print a message.

hello.sh

```
1  #!/bin/bash
2  # This is my first shell script
3  echo "Hello, DevOps!"
4  echo "Welcome to Shell Scripting"
```

Output

Terminal

```
anush@linux:~$ ./hello.sh
Hello, DevOps!
Welcome to Shell Scripting
```

Benefits of Shell Scripting

1. Automate repetitive tasks
2. Save time and reduce manual work
3. Easy to learn and use
4. Useful for system administration
5. Widely used in DevOps and Cloud
6. Helps in monitoring, backup, deployment
7. Excellent for interview preparation

Common Shells in Linux

- **sh** – Bourne Shell (old)
- **bash** – Bourne Again Shell (default)
- **ksh** – Korn Shell
- **csh** – C Shell
- **zsh** – Z Shell (advanced features)

Run the Script

1. Save the file: hello.sh
2. Give execute permission:
   `chmod +x hello.sh`
3. Run the script:
   `./hello.sh`

Key Points

- ✓ Shell is a command line interface
- ✓ Shell scripting automates tasks
- ✓ Bash is the most commonly used shell
- ✓ A shell script is a plain text file
- ✓ Useful for DevOps and system administration

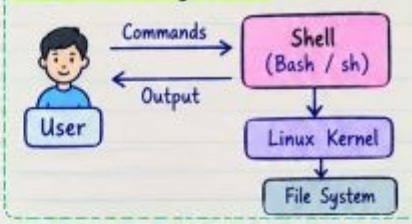
Manjunatha HG

2. Linux Shell & bash Basics

2.1 What is Linux Shell ?

- A shell is a command line interface that allows users to interact with the Linux operating system.
- It takes commands from the user, interprets them and passes them to the kernel for execution.
- Shell provides a way to run commands either interactively or through scripts.

Shell Working Flow



2.2 What is Bash ?

- Bash (Bourne Again Shell) is the most commonly used shell in Linux.
- It is an enhanced version of the Bourne shell.
- Bash supports command history, tab completion, scripting and many useful features.
- It is the default shell in most Linux distributions like Ubuntu, CentOS, RHEL, Amazon Linux etc.

2.3 Types of Shells in Linux

Shell	Full Form	Features
sh	Bourne Shell	Original shell, basic features
bash	Bourne Again Shell	Most common, default shell, advanced features
ksh	Korn Shell	Developed by AT&T, used in many systems
csh	C Shell	C-like syntax, less common
zsh	Z Shell	Advanced features, widely used in modern systems

2.4 Shebang (#!)

- The first line of a shell script is called shebang (#!).
- It tells the system which shell should be used to execute the script.
- For bash scripting, we use:

`#!/bin/bash` ⟷ This is bash shell

2.5 Check Current Shell

- To check which shell you are currently using:

```
rakesh@linux:~$ echo $SHELL
/bin/bash
```

2.6 Check Available Shells

- List all available shells on your system:

```
rakesh@linux:~$ cat /etc/shells
/bin/sh
/bin/bash
/bin/ksh
/bin/csh
/bin/zsh
```

2.7 Common Terminal Commands

Command	Description
pwd	Print current directory
ls	List files and directories
cd	Change directory
whoami	Show current user
hostname	Show system hostname
date	Show current date and time
man <command>	Show manual/help for a command
clear	Clear terminal screen

2.8 Script Structure

```
#!/bin/bash          → Shebang
# This is a comment  → Comment
# Script logic goes here
echo "Hello from my script!"   → Commands
```

2.9 Execute a Script

1. Save the script in a file (e.g. script.sh)
2. Give execute permission: chmod +x script.sh
3. Run the script: ./script.sh
4. Or run using bash explicitly: bash script.sh

2.10 Key Points

- ✓ Shell is a command line interface.
- ✓ Bash is the most commonly used shell.
- ✓ Shebang tells which shell to use.
- ✓ Shell scripts automate repetitive tasks.

Important Notes

- ◆ Script needs execute permission.
- ◆ Use comments to make script readable.
- ◆ Shell supports variables, conditions, loops and functions.
- ◆ Useful for system administration and DevOps.

Manjunatha HG

3. Variables & User Input

3.1 What are Variables ?

- Variables are used to store data in shell scripts.
- They can store strings, numbers and command output.
- No need to declare variable type in shell.
- Variable names can contain letters, numbers and underscore (_).
- Variable names should not start with a number.

3.2 Create and Use Variables

script.sh

```
1  name="Anush"     # string variable
2  age=28           # number variable
3  echo "My name is $name"
4  echo "My age is $age"
```

3.3 Types of Variables

Type	Example	Description
String Variable	name="DevOps"	Stores text value
Number Variable	count=10	Stores numeric value
Command Substitution	date=$(date)	Stores output of a command
Environment Variable	echo $PATH	Predefined by the system
Read-Only Variable	readonly PI=3.14	Cannot be changed

3.5 User Input using read

- read command is used to take input from the user and store it in a variable.
- Useful for interactive scripts.

input.sh

```
1  #!/bin/bash
2  read -p "Enter your name: " name
3  read -p "Enter your age: " age
4  echo "Hello $name, you are $age years old."
```

3.6 Command Line Arguments

- You can pass values to a script while running it.
- These are called command line arguments.
- $0 – script name
- $1 – first argument
- $2 – second argument
- $# – total number of arguments
- $@ – all arguments

Variable Syntax

No spaces around = → VARIABLE_NAME = value

Use variable with $ → echo $VARIABLE_NAME

Output

Terminal

```
anush@linux:~$ bash script.sh
My name is Anush
My age is 28
```

3.4 Environment Variables

- Environment variables are global variables available to all users and processes.
- Common environment variables:

Variable	Description
PATH	Directory paths for executable files
HOME	Home directory of current user
USER	Current username
SHELL	Current shell path
PWD	Current working directory

- Check environment variable:

Terminal

```
anush@linux:~$ echo $PATH
/usr/local/bin:/usr/bin:/bin:...
```

Output

Terminal

```
anush@linux:~$ bash input.sh
Enter your name: Anush
Enter your age: 28
Hello Anush, you are 28 years old.
```

Example: args.sh

```
1  #!/bin/bash
2  echo "Script name: $0"
3  echo "First argument: $1"
4  echo "Second argument: $2"
5  echo "Total arguments: $#"
6  echo "All arguments: $@"
```

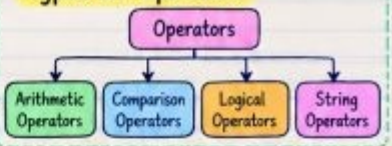
Manjunatha HG

4. Operators in Shell Scripting

4.1 What are Operators ?

- Operators are special symbols used to perform operations on variables and values.
- They help in arithmetic calculations, comparisons, logical operations and string operations.
- Operators are widely used in conditions, loops and automation scripts.

Types of Operators



4.2 Arithmetic Operators

- Used to perform mathematical calculations.

Operator	Description	Example	Result
+	Addition	$((5 + 3))	8
-	Subtraction	$((5 - 3))	2
*	Multiplication	$((5 * 3))	15
/	Division	$((10 / 3))	3
%	Modulus (Remainder)	$((10 % 3))	1

Arithmetic Operator Example

arithmetic.sh

```
#!/bin/bash
a=10
b=3
echo "Addition: $((a + b))"
echo "Subtraction: $((a - b))"
echo "Multiplication: $((a * b))"
echo "Division: $((a / b))"
echo "Modulus: $((a % b))"
```

Output

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3
Modulus: 1
```

4.3 Comparison Operators

- Used to compare two values. Mostly used in if conditions.

Operator	Description	Example	Result
-eq	Equal to	[5 -eq 5]	true
-ne	Not equal to	[5 -ne 3]	true
-gt	Greater than	[5 -gt 3]	true
-lt	Less than	[3 -lt 5]	true
-ge	Greater than or equal	[5 -ge 5]	true
-le	Less than or equal	[3 -le 5]	true

Comparison Example

compare.sh

```
#!/bin/bash
a=10
b=20
if [ $a -lt $b ]; then
    echo "$a is less than $b"
else
    echo "$a is not less than $b"
fi
```

Output

```
10 is less than 20
```

4.4 Logical Operators

- Used to combine multiple conditions.

Operator	Description	Example
&&	AND (Both true)	[$a -gt 5] && [$b -lt 10]
||	OR (Any one true)	[$a -gt 5] || [$b -lt 10]
!	NOT (Opposite)	[! $a -gt 5]

Logical Operator Example

logical.sh

```
#!/bin/bash
a=8
b=12
if [ $a -gt 5 ] && [ $b -gt 10 ]; then
    echo "Both conditions are true"
else
    echo "Condition failed"
fi
```

Output

```
Both conditions are true
```

4.5 String Operators

- Used to perform operations on strings.

Operator	Description	Example
=	Equal to	["$a" = "$b"]
!=	Not equal to	["$a" != "$b"]
-z	String is empty	[-z "$a"]
-n	String is not empty	[-n "$a"]

String Operator Example

string.sh

```
#!/bin/bash
name="Anush"
if [ -n "$name" ]; then
    echo "Name is not empty: $name"
else
    echo "Name is empty"
fi
```

Output

```
Name is not empty: Anush
```

4.6 Key Points

- ✔ Arithmetic operators are used with $(( ))
- ✔ Comparison operators are used in if conditions
- ✔ Logical operators combine multiple conditions
- ✔ String operators work on text values
- ✔ Operators are widely used in shell scripts for automation

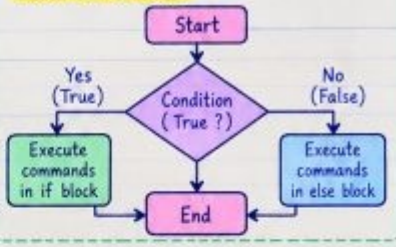
Manjunatha HG

5. Conditional Statements in Shell Scripting

5.1 What are Conditional Statements ?

- Conditional statements are used to make decisions in a shell script.
- They allow the script to execute different commands based on a condition (true or false).
- Mainly used: if, if-else, elif, nested if and case.
- Conditions are evaluated using comparison operators.

Flow of if-else



5.2 if Statement (Basic Syntax)

if_example.sh

```
#!/bin/bash
if [ condition ]; then
    # commands to execute if true
fi
```

Example: Check if a file exists

file_check.sh

```
#!/bin/bash
if [ -f "data_txt" ]; then
    echo "File exists"
fi
```

Output

```
$ bash file_check.sh
File exists
```

5.3 if-else Statement

if_else.sh

```
#!/bin/bash
if [ condition ]; then
    # commands if true
else
    # commands if false
fi
```

Example: Check if a directory exists

dir_check.sh

```
#!/bin/bash
if [ -d "/home/anush" ]; then
    echo "Directory exists"
else
    echo "Directory not found"
fi
```

Output (if exists)

```
$ bash dir_check.sh
Directory exists
```

Output (if not exists)

```
$ bash dir_check.sh
Directory not found
```

5.4 if-elif-else Statement

if_elif.sh

```
#!/bin/bash
age=18
if [ $age -lt 18 ]; then
    echo "Minor"
elif [ $age -eq 18 ]; then
    echo "Adult (18 years)"
else
    echo "Adult"
fi
```

Example: Check number

check_number.sh

```
#!/bin/bash
num=0
if [ $num -gt 0 ]; then
    echo "Positive"
elif [ $num -lt 0 ]; then
    echo "Negative"
else
    echo "Zero"
fi
```

Output

```
$ bash check_number.sh
Zero
```

5.5 Nested if Statement

nested_if.sh

```
#!/bin/bash
num=10
if [ $num -gt 0 ]; then
    if [ $num -gt 5 ]; then
        echo "Number is greater than 5"
    else
        echo "Number is 1 to 5"
    fi
else
    echo "Number is zero or negative"
fi
```

5.6 Common Conditions (File Test Operators)

Operator	Description	Example
-f	File exists and is a regular file	[-f file.txt]
-d	Directory exists	[-d /home/user]
-e	File or directory exists	[-e file.txt]
-s	File exists and is not empty	[-s file.txt]
-r	File is readable	[-r file.txt]
-w	File is writable	[-w file.txt]
-x	File is executable	[-x file.sh]

Manjunatha HG

6. Case Statements in Shell Scripting

6.1 What is Case Statement ?

- Case statement is used to execute a block of commands based on a match with a value.
- It is an alternative to multiple if-else conditions.
- Mostly used for menu-driven scripts.
- It compares a value with multiple patterns and executes the matching block.
- More readable and easier to maintain than multiple if-else.

Syntax of Case Statement

```
case variable in
  pattern1)
     commands ;;
  pattern2)
     commands ;;
  pattern3)
     commands ;;
  *)
     default commands ;;
esac
```

- `case variable in` — Keyword to start case statement
- `pattern1)` — Pattern to match the value
- `commands ;;` — Commands to execute if pattern matches
- `*)` — Default case (if no pattern matches)
- `esac` — End of case statement

6.2 Simple Case Statement Example

simple_case.sh

```
#!/bin/bash
echo "Enter your choice (start/stop/restart):"
read choice
case $choice in
   start)
      echo "Starting the service..." ;;
   stop)
      echo "Stopping the service..." ;;
   restart)
      echo "Restarting the service..." ;;
   *)
      echo "Invalid choice!" ;;
esac
```

Output

```
$ bash simple_case.sh
Enter your choice (start/stop/restart):
start
Starting the service...
```

6.3 Case with Multiple Patterns

multiple_pattern.sh

```
#!/bin/bash
echo "Enter environment (dev/qa/stg/prod):"
read env
case $env in
   dev|development)
      echo "Deploying to Development" ;;
   qa|test)
      echo "Deploying to QA" ;;
   stg|stage)
      echo "Deploying to Staging" ;;
   prod|production)
      echo "Deploying to Production" ;;
   *)
      echo "Invalid environment!" ;;
esac
```

6.4 Menu Driven Script (Practical Example)

menu.sh

```
#!/bin/bash
echo "===== System Menu ====="
echo "1. Check Disk Space"
echo "2. Check Memory Usage"
echo "3. Check Current User"
echo "4. Show Running Services"
echo "5. Exit"
read -p "Enter your choice (1-5): " choice
case $choice in
   1) df -h ;;
   2) free -h ;;
   3) whoami ;;
   4) systemctl list-units --type=service --state=running ;;
   5) echo "Exiting..." ;;
   *) echo "Invalid choice!" ;;
esac
```

Example Output (Option 1)

```
$ bash menu.sh
===== System Menu =====
1. Check Disk Space
2. Check Memory Usage
3. Check Current User
4. Show Running Services
5. Exit
Enter your choice (1-5): 1
Filesystem      Size  Used Avail Use% Mounted on
/dev/xvda1       30G   12G   18G  40% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
```

6.5 Key Points

- ✓ Use case when you have multiple conditions.
- ✓ Each pattern ends with ;;
- ✓ Use * as default case.
- ✓ You can match multiple values using | (OR).
- ✓ Case statement is more readable than multiple if-else.
- ✓ Useful for menu-driven and automation scripts.

6.6 Real-World Use Cases

1. Menu-driven utility scripts
2. Environment selection (dev/qa/prod)
3. Service management (start/stop/restart)
4. File operations based on file type
5. User input handling in automation scripts.

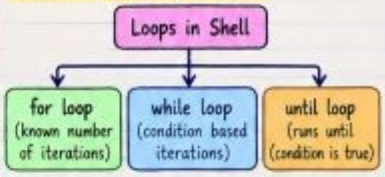
Manjunatha HG

7. Loops in Shell Scripting

7.1 What are Loops ?

- Loops are used to execute a set of commands multiple times.
- They help in automating repetitive tasks.
- Mainly used for for loop, while loop and until loop.
- Useful in system administration, file operations, log analysis and DevOps automation.

Types of Loops in Shell



7.2 for Loop

- Used when the number of iterations is known.
- It iterates over a list of items (numbers, strings, files etc).

Syntax

```
for variable in list
 do
    commands
done
```

Example: Print numbers 1 to 5

for_loop.sh

```
#!/bin/bash
for i in 1 2 3 4 5
do
    echo "Number: $i"
done
```

Output

```
$ bash for_loop.sh
Number: 1
Number: 3
Number: 4
Number: 5
```

7.3 for Loop with List of Items

- Iterate over strings, files or command output.

Example: Loop over a list of users

user_loop.sh

```
#!/bin/bash
for user in anush rahul admin test
do
    echo "User: $user"
done
```

Output

```
$ bash user_loop.sh
User: anush
User: rahul
User: admin
User: test
```

Example: Loop over files in a directory

file_loop.sh

```
#!/bin/bash
for file in *.txt
do
    echo "File: $file"
done
```

Output (example)

```
$ bash file_loop.sh
File: data.txt
File: demo.txt
File: notes.txt
```

7.4 while Loop

- Executes commands as long as the condition is true.
- Best used when the number of iterations is not known.

Syntax

```
while [ condition ]
 do
    commands
done
```

Example: Print numbers 1 to 5 using while

while_loop.sh

```
#!/bin/bash
i=1
while [ $i -le 5 ]
do
    echo "Number: $i"
    i=$((i+1))
done
```

Output

```
$ bash while_loop.sh
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

7.5 until Loop

- Executes commands until the condition becomes true.
- It is opposite to while loop.

Syntax

```
until [ condition ]
 do
    commands
done
```

Example: Print numbers 1 to 5 using until

until_loop.sh

```
#!/bin/bash
i=1
until [ $i -gt 5 ]
do
    echo "Number: $i"
    i=$((i+1))
done
```

Output

```
$ bash until_loop.sh
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

7.6 Key Points

- ✓ Use for loop when the number of iterations is known.
- ✓ Use while loop when the condition is true.
- ✓ Use until loop when the condition becomes true.
- ✓ Loops help in automating repetitive tasks.
- ✓ Useful for file operations, system administration and DevOps.

7.7 Real-World Use Cases

1. Loop through server list for health checks
2. Process log files line by line
3. Automate backups of multiple files
4. Manage multiple users and directories
5. Useful in DevOps and automation scripts

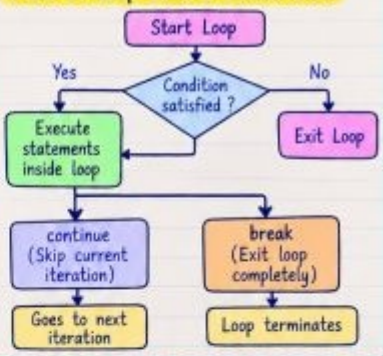
Manjunatha HG

8. Loop Control Statements

8.1 What are Loop Control Statements ?

- Loop control statements are used to change the normal flow of loop execution.
- They are mainly of two types: break and continue.
- break is used to exit the loop completely.
- continue is used to skip the current iteration and move to the next iteration.
- These are useful when you want to apply specific conditions inside loops.

Flow of Loop Control Statements



8.2 break Statement

- Used to terminate the loop completely.
- Once break is executed, the loop is stopped and control moves to the next statement after the loop.

Example: break after number 3

break_example.sh

```
#!/bin/bash
for i in 1 2 3 4 5
do
    if [ $i -eq 3 ]; then
        break
    fi
    echo "Number: $i"
done
```

Output

```
$ bash break_example.sh
Number: 1
Number: 2
```

When i = 3, break is executed, so the loop stops and 3, 4, 5 are not printed.

8.3 continue Statement

- Used to skip the current iteration.
- When continue is executed, the remaining commands in the current iteration are skipped and the loop moves to the next iteration.

Example: skip even numbers

continue_example.sh

```
#!/bin/bash
for i in 1 2 3 4 5
do
    if [ $i -eq 2 ] || [ $i -eq 4 ]; then
        continue
    fi
    echo "Number: $i"
done
```

Output

```
$ bash continue_example.sh
Number: 1
Number: 3
Number: 5
```

When i = 2 or 4, continue is executed, so those numbers are skipped.

8.4 Nested Loop with break and continue

- Loop control statements can also be used in nested loops.

Example: nested loop

nested_loop.sh

```
#!/bin/bash
for i in 1 2 3
do
    for j in 1 2 3
    do
        if [ $j -eq 2 ]; then
            continue  # skip j = 2
        fi
        if [ $i -eq 3 ] && [ $j -eq 3 ]; then
            break     # exit inner loop
        fi
        echo "i=$i, j=$j"
    done
done
```

Output

```
$ bash nested_loop.sh
i=1, j=1
i=1, j=3
i=2, j=1
i=2, j=3
i=3, j=1
```

j = 2 is skipped due to continue.
For i = 3, j = 3 stops due to break.

8.5 Key Points

- ✓ break terminates the loop completely.
- ✓ continue skips the current iteration.
- ✓ Useful in nested loops.
- ✓ Helps in handling specific conditions inside loops.
- ✓ Commonly used in automation scripts, file processing and log analysis.

8.6 Real-World Use Cases

1. Stop processing when an error occurs.
2. Skip specific files while iterating.
3. Process only required records.
4. Validate data in loops.
5. Useful in DevOps and automation scripts.

Manjunatha HG

9. Functions in Shell Scripting

9.1 What are Functions ?

- A function is a block of reusable code that performs a specific task.
- Functions help to organize the script, reduce code repetition and make the script more readable.
- A function is defined once and can be called multiple times.
- Functions can accept parameters and can also return a value (exit status).
- Widely used in automation scripts, DevOps scripts and large shell scripts.

Benefits of Functions

- ✔ Code reusability
- ✔ Better organization
- ✔ Easy to maintain
- ✔ Reduces code duplication
- ✔ Helps in debugging
- ✔ Useful for large scripts
- ✔ Supports parameters and return status

9.2 Function Syntax

Method 1 (Recommended)

```
function_name() {
    commands
}
```

Method 2 (Also valid)

```
function function_name {
    commands
}
```

Function Flow



9.3 Simple Function Example

simple_function.sh

```
#!/bin/bash
greet() {
    echo "Hello, Welcome to Shell Scripting!"
}
# Calling the function
greet
```

Output

```
$ bash simple_function.sh
Hello, Welcome to
Shell Scripting!
```

9.4 Function with Parameters

function_parameter.sh

```
#!/bin/bash
greet_user() {
    echo "Hello $1, Welcome to $2!"
}
# Calling the function with arguments
greet_user "Anush" "DevOps"
```

Output

```
$ bash function_parameter.sh
Hello Anush, Welcome to DevOps!
```

9.5 Function with Return Status

function_return.sh

```
#!/bin/bash
check_number() {
    if [ $1 -gt 0 ]; then
        return 0    # success
    else
        return 1    # failure
    fi
}
check_number 5
echo "Exit status: $?"

check_number -3
echo "Exit status: $?"
```

Output

```
$ bash function_return.sh
Exit status: 0
Exit status: 1
```

- Functions return an exit status (0 for success, non-zero for failure).
- We can check the return status using $?.
- Return values are numbers, not strings.

9.6 Function with Local Variables

local_variable.sh

```
#!/bin/bash
show_name() {
    local name="Anush"  # local variable
    echo "Inside function: $name"
}
show_name
echo "Outside function: $name"
```

Output

```
$ bash local_variable.sh
Inside function: Anush
Outside function:
```

- Variables declared with local are only accessible inside the function.
- Helps to avoid variable conflict.

9.7 Key Points

- ✔ Use functions to avoid code repetition.
- ✔ Functions can accept parameters like $1, $2 etc.
- ✔ Functions can return exit status using return.
- ✔ Use local variables to avoid conflicts.
- ✔ Functions make scripts modular and easy to maintain.

9.8 Real-World Use Cases

1. Create reusable functions for deployment steps
2. Functions for system health checks (CPU, Memory, Disk)
3. Log processing and log analysis
4. Backup and cleanup tasks
5. Used in DevOps automation scripts

Manjunatha HG

10. Arrays & String Manipulation in Shell Scripting

10.1 What are Arrays ?

- An array is a collection of multiple values stored in a single variable.
- Arrays help to store and process lists of data like names, files, servers etc.
- Bash supports indexed arrays (numerical index).
- Array elements can be accessed using index starting from 0.
- Useful for iterating over multiple values in scripts.

10.2 Array Syntax

Declaration
```
array_name=(value1 value2 value3 ....)
```

Access Element
```
${array_name[index]}
```

Get All Elements
```
${array_name[@]}
${array_name[*]}
```

Get Array Length
```
${#array_name[@]}
```

10.3 Array Example

array_example.sh
```
#!/bin/bash
# Define array
fruits=("Apple" "Banana" "Orange" "Mango")

# Access elements
echo "First fruit: ${fruits[0]}"
echo "All fruits: ${fruits[@]}"
echo "Total fruits: ${#fruits[@]}"
```

Output
```
$ bash array_example.sh
First fruit: Apple
All fruits: Apple Banana
Orange Mango
Total fruits: 4
```

10.4 Loop through Array

array_loop.sh
```
#!/bin/bash
fruits=("Apple" "Banana" "Orange" "Mango")

for fruit in "${fruits[@]}"
do
    echo "Fruit: $fruit"
done
```

Output
```
$ bash array_loop.sh
Fruit: Apple
Fruit: Banana
Fruit: Orange
Fruit: Mango
```

10.5 Array with Index

array_index.sh
```
#!/bin/bash
fruits=("Apple" "Banana" "Orange" "Mango")

for i in "${!fruits[@]}" # get all indexes
do
    echo "Index $i: ${fruits[$i]}"
done
```

Output
```
$ bash array_index.sh
Index 0: Apple
Index 1: Banana
Index 2: Orange
Index 3: Mango
```

10.6 String Manipulation

- Strings are used to store and manipulate text.
- Bash provides many string operations like length, substring, replace, etc.
- Useful for working with file names, logs, user input and environment variables.

Common String Operations

Operation	Example
String length	${#str}
Substring (from position)	${str:0:4}
Remove prefix	${str#pattern}
Remove suffix	${str%pattern}
Replace string	${str/old/new}
Convert to uppercase	${str^^}
Convert to lowercase	${str,,}

10.7 String Example

string_example.sh
```
#!/bin/bash
str="DevOpsEngineer"

echo "Original String: $str"
echo "Length: ${#str}"
echo "Substring (0-6): ${str:0:6}"
echo "Remove prefix (Dev): ${str#Dev}"
echo "Replace (DevOps -> Cloud):
      ${str/DevOps/Cloud}"
echo "Uppercase: ${str^^}"
echo "Lowercase: ${str,,}"
```

Output
```
$ bash string_example.sh
Original String: DevOpsEngineer
Length: 15
Substring (0-6): DevOps
Remove prefix (Dev): OpsEngineer
Replace (DevOps -> Cloud):
CloudEngineer
Uppercase: DEVOPSENGINEER
Lowercase: devopsengineer
```

10.8 Taking User Input and Storing in Array

user_array.sh
```
#!/bin/bash
echo "Enter 3 server names:"
read -a servers

echo "You entered:"
for server in "${servers[@]}"
do
    echo "Server: $server"
done
```

Output
```
$ bash user_array.sh
Enter 3 server names:
web1 web2 web3
You entered:
Server: web1
Server: web2
Server: web3
```

10.9 Key Points

- ✓ Bash supports indexed arrays (start index 0).
- ✓ Use ${array[@]} to get all elements.
- ✓ Use ${#array[@]} to get array length.
- ✓ Arrays are useful for processing lists of data.
- ✓ String operations help in file handling, log analysis and text processing.
- ✓ Commonly used in automation scripts and DevOps tasks.

10.10 Real-World Use Cases

1. Store list of servers and perform operations in loop.
2. Read files and store lines in an array.
3. Process file names and extract information.
4. Handle user input and validate multiple values.
5. Log analysis and text processing.
6. Useful in DevOps automation scripts.

Manjunatha HG

13. Text Processing Commands in Shell Scripting

13.1 What are Text Processing Commands ?

- Text processing commands are used to search, filter, extract, modify and analyze text data from files or command output.
- Widely used in log analysis, monitoring, automation scripts and DevOps tasks.
- Common commands: grep, awk, sed, cut, sort, uniq, head, tail, tr, wc, etc.

13.2 Common Text Processing Commands

Command	Description
grep	Search for a pattern
awk	Pattern scanning and processing
sed	Stream editor to modify text
cut	Extract specific columns
sort	Sort lines of text
uniq	Remove duplicate lines
head	Show first N lines
tail	Show last N lines
tr	Translate or delete characters
wc	Count lines, words and characters

13.3 grep Command (Search Pattern)

grep_example.sh

```
#!/bin/bash
# Search for 'ERROR' in log file
grep "ERROR" app.log
# Ignore case
grep -i "error" app.log
# Show line number
grep -n "ERROR" app.log
# Show lines before and after match
grep -C 2 "ERROR" app.log
```

Output

```
$ grep "ERROR" app.log
ERROR: Connection failed
ERROR: Database not found

$ grep -n "ERROR" app.log
12:ERROR: Connection failed
45:ERROR: Database not found
```

Common grep Options

Option	Description
-i	Ignore case
-n	Show line number
-v	Invert match
-r	Search recursively
-C N	Show N lines before and after match

13.4 awk Command (Pattern Scanning)

awk_example.sh

```
#!/bin/bash
# Print first column
awk '{print $1}' app.log
# Print specific column (e.g. 3rd)
awk '{print $3}' app.log
# Filter rows with condition
awk '$3 > 100 {print $1, $3}' access.log
```

Output

```
$ awk '{print $1}' app.log
INFO
ERROR
WARN
$ awk '$3 > 100 {print $1, $3}'
user1 120
user3 450
```

awk Key Points

- Works on columns (fields).
- $1, $2, $3 are column numbers.
- Useful for log analysis.
- Supports conditions, loops and calculations.
- Very powerful for DevOps automation.

13.5 sed Command (Stream Editor)

sed_example.sh

```
#!/bin/bash
# Replace text in file (output to screen)
sed 's/old/new/g' file.txt
# Replace text and save to new file
sed 's/old/new/g' file.txt > newfile.txt
# Delete a specific line (e.g. line 3)
sed '3d' file.txt
```

Output

```
$ sed 's/old/new/g' file.txt
This is new line
Another new line
$ sed '3d' file.txt
Line 1
Line 2
Line 4
```

sed Common Use Cases

- Find and replace text.
- Delete specific lines.
- Insert text.
- Modify configuration files.
- Useful in deployment scripts to update values dynamically.

13.6 cut Command (Extract Columns)

cut_example.sh

```
#!/bin/bash
# Extract first column (default delimiter)
cut -d' ' -f1 users.txt
# Extract specific column using :
cut -d':' -f1 /etc/passwd
# Extract multiple columns
cut -d',' -f1,3 data.csv
```

Output

```
$ cut -d' ' -f1 users.txt
anush
john
devops
$ cut -d':' -f1 /etc/passwd
root
anush
ec2-user
```

cut Key Points

- Used to extract specific fields from text.
- Specify delimiter with -d.
- Specify field number using -f.
- Useful for working with CSV, logs and configuration files.

13.7 sort Command (Sort Lines)

sort_example.sh

```
#!/bin/bash
# Sort file in ascending order
sort names.txt
# Sort in descending order
sort -r names.txt
# Sort numerically
sort -n numbers.txt
```

Output

```
$ sort names.txt
anush
devops
john
kubernetes
$ sort -n numbers.txt
10
50
100
```

13.8 uniq Command (Remove Duplicates)

uniq_example.sh

```
#!/bin/bash
# Remove duplicate lines
sort names.txt | uniq
# Show only duplicate lines
sort names.txt | uniq -d
# Show count of duplicates
sort names.txt | uniq -c
```

Output

```
$ sort names.txt | uniq
anush
devops
john
$ sort names.txt | uniq -d
anush
$ sort names.txt | uniq -c
  2  2 anush
  1  devops
```

13.9 Key Points

- Used to search, filter, extract and modify text data.
- Very useful for log analysis, file processing and automation.
- Can be combined using pipes (|) for powerful operations.
- Commonly used in DevOps, monitoring and production scripts.

13.10 Real-World Use Cases

1. Analyze and filter application logs.
2. Extract specific data from large files (CSV, logs).
3. Clean and transform text data.
4. Monitor system logs in real-time.
5. Update configuration files using sed.
6. Useful in DevOps automation and deployme

Manjunatha HG

14. Exit Status & Error Handling in Shell Scripting

14.1 What is Exit Status ?

- Every command in Linux returns an exit status.
- Exit status is a number that indicates whether a command was successful or failed.
- 0 → Success
- Non-zero → Failure (1, 2, 126, 127, etc.)
- We can check the exit status using $? variable.
- Useful for error handling, debugging and automation scripts.

14.2 Common Exit Status Codes

Code	Meaning
0	Success
1	General error
2	Misuse of shell builtins
126	Command found but not executable
127	Command not found
130	Script terminated (Ctrl + C)
137	Killed (out of memory)
255	Exit status out of range

14.3 Check Exit Status Example

exit_status.sh

```
#!/bin/bash
# Check exit status of a command
ls /tmp/demo
echo "Exit status: $?"

ls /tmp/notfound
echo "Exit status: $?"
```

Output

```
$ bash exit_status.sh
file.txt
Exit status: 0

ls: cannot access '/tmp/notfound': No such file or directory
Exit status: 2
```

14.4 Using exit Command

exit_example.sh

```
#!/bin/bash
if [ -f "app.conf" ]; then
    echo "Config file found!"
    exit 0
else
    echo "Config file not found!"
    exit 1
fi
```

14.5 set -e (Exit on Error)

set_e_example.sh

```
#!/bin/bash
set -e    # Exit immediately if error
echo "Step 1: List files"
ls /tmp/demo
echo "Step 2: Invalid command"
invalid_command
echo "This line will not execute"
```

Output

```
$ bash set_e_example.sh
Step 1: List files
file.txt
Step 2: Invalid command
set_e_example.sh: line 6: invalid_command: command not found
```

14.6 set -u (Error on Undefined Variables)

set_u_example.sh

```
#!/bin/bash
set -u  # Treat undefined variables as error
echo "Username: $USER"
echo "Home: $HOME"
echo "Undefined: $UNDEF"
```

Output

```
$ bash set_u_example.sh
Username: rakesh
Home: /home/rakesh
set_u_example.sh: line 5: UNDEF: unbound variable.
```

14.7 set -o pipefail (Catch Errors in Pipeline)

pipefail_example.sh

```
#!/bin/bash
set -o pipefail
cat /tmp/notfound | grep "error"
echo "Exit status: $?"
```

Output

```
$ bash pipefail_example.sh
cat: /tmp/notfound: No such file or directory
Exit status: 1
```

- Without pipefail, pipeline returns exit status of last command only.
- With pipefail, pipeline fails if any command in the pipeline fails.

14.8 Using trap for Error Handling

trap_example.sh

```
#!/bin/bash
trap 'echo "Error occurred at line $LINENO"; exit 1' ERR
echo "Starting script..."
ls /tmp/demo
invalid_command
echo "This will not run"
```

Output

```
$ bash trap_example.sh
Starting script...
Error occurred at line 5
trap_example.sh: line 5: invalid_command: command not found
```

- trap is used to catch errors and perform cleanup actions.
- Here ERR trap will run when any command fails.

14.9 Debugging Shell Scripts

debug_example.sh

```
#!/bin/bash
set -x  # Print each command
echo "Debug mode enabled"
ls /tmp/demo
set +x  # Disable debug mode
```

Output

```
$ bash debug_example.sh
+ echo 'Debug mode enabled'
Debug mode enabled
+ ls /tmp/demo
file.txt
+ set +x
```

- set -x helps to print each command before executing it.
- Useful for troubleshooting script issues.

14.10 Key Points

- ✅ Use $? to check the exit status of a command.
- ✅ 0 means success, non-zero means failure.
- ✅ Use exit to stop script with a specific status.
- ✅ Use set -e to exit on error.
- ✅ Use set -u to catch undefined variables.
- ✅ Use set -o pipefail to handle errors in pipelines.
- ✅ Use trap to handle errors and perform cleanup.
- ✅ Use set -x for debugging.
- ✅ Important for DevOps automation and production scripts.

14.11 Real-World Use Cases

1. Validate command execution before proceeding to next step.
2. Handle errors in deployment scripts.
3. Check if required files or directories exist.
4. Clean up resources on failure.
5. Ensure reliable automation in DevOps pipelines.

14.12 Sample Script: Safe Deployment with Error Handling

deploy.sh

```
#!/bin/bash
set -e
trap 'echo "Deployment failed at line $LINENO"; exit 1' ERR
echo "Deploying application..."
# deployment commands here
echo "Deployment completed successfully!"
```

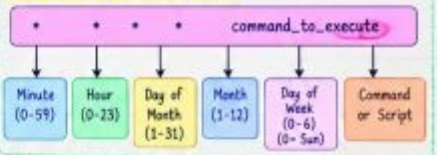
Manjunatha HG

15. Cron Jobs & Task Scheduling in Shell Scripting

15.1 What are Cron Jobs ?

- Cron is a time-based job scheduler in Linux.
- It allows us to run commands or shell scripts automatically at a specified date and time.
- Useful for automation, backups, log cleanup, monitoring and periodic tasks.
- Cron jobs are managed using crontab.

15.2 Crontab Syntax



15.3 Special Characters in Cron

Character	Description
*	Matches all values
,	Multiple values (e.g. 1,2,5)
-	Range of values (e.g. 1-5)
/	Step values (e.g. */5, 1-10/2)

15.4 Common Cron Examples

Schedule	Cron Expression	Description
Every minute	* * * * *	Run every minute
Every 5 minutes	*/5 * * * *	Run every 5 minutes
Every hour	0 * * * *	Run at 0th minute every hour
Every day at 2 AM	0 2 * * *	Run daily at 2 AM
Every week on Sunday	0 2 * * 0	Run every Sunday at 2 AM
Every month on 1st	0 2 1 * *	Run on 1st of every month
Every weekday at 9 AM	0 9 * * 1-5	Run on Mon-Fri at 9 AM

15.5 Basic Crontab Commands

- ✅ `crontab -e` → Edit crontab
- ✅ `crontab -l` → List existing cron jobs
- ✅ `crontab -r` → Remove all cron jobs
- ✅ `crontab -u <user>` → Edit for specific user

Add to Crontab

```
$ crontab -e
*/5 * * * * /home/anush/cron_simple.sh
```

15.6 Create a Simple Cron Job

cron_simple.sh

```
#!/bin/bash
# This script will run every 5 minutes
echo "Cron job executed at: $(date)"
echo "Hello from cron!" >> /tmp/cron.log
```

Add to Crontab

```
$ crontab -e
*/5 * * * * /home/anush/cron_simple.sh
```

Verify Cron Job

```
$ crontab -l
*/5 * * * * /home/anush/cron_simple.sh
```

15.7 Cron Job for System Backup

backup.sh

```
#!/bin/bash
# Create backup of /var/log directory
DATE=$(date +%Y-%m-%d)
tar -czf /backup/logs_${DATE}.tar.gz /var/log
echo "Backup completed: logs_${DATE}.tar.gz"
```

Add to Crontab (Daily at 2 AM)

```
$ crontab -e
0 2 * * * /home/anush/backup.sh
```

Output (After Execution)

```
$ ls -lh /backup
total 4.0M
-rw-r--r-- 1 anush anush 4.0M
Oct 20 02:00 logs_2026-10-20.tar.gz
```

15.8 Cron Job for Log Cleanup

cleanup.sh

```
#!/bin/bash
# Delete log files older than 7 days
find /var/log -name "*.log" -type f -mtime +7 -delete
echo "Old log files cleaned: $(date)" >> /tmp/cleanup.log
```

Add to Crontab (Daily at 1 AM)

```
$ crontab -e
0 1 * * * /home/anush/cleanup.sh
```

15.9 Cron Job for Monitoring (Disk Usage Alert)

disk_alert.sh

```
#!/bin/bash
THRESHOLD=80
USAGE=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ $USAGE -gt $THRESHOLD ]; then
    echo "Disk usage is high: ${USAGE}%" | mail -s "Disk Alert" anush@gmail.com
fi
```

Add to Crontab (Every Hour)

```
$ crontab -e
0 * * * * /home/anush/disk_alert.sh
```

15.10 Key Points

- ✅ Use absolute paths in cron jobs.
- ✅ Set proper permissions for scripts.
- ✅ Use output redirection for logs.
- ✅ Test scripts manually before adding to crontab.
- ✅ Use crontab -l to verify scheduled jobs.
- ✅ Very useful for automation, backups, monitoring and maintenance.

15.11 Real-World Use Cases

1. Automated log cleanup.
2. Database backup scripts.
3. System health monitoring (CPU, Disk, Memory).
4. Send email alerts for failures.
5. Regular file processing and data updates.
6. Useful in DevOps automation and production environments.

Manjunatha HC

16. System Monitoring Scripts in Shell Scripting

16.1 Introduction

- System monitoring helps us to check the health of a server (CPU, Memory, Disk, Services, etc.).
- We can use shell scripts to automate monitoring and send alerts.
- Useful for DevOps, production environments and server maintenance.
- We can log the output, send email alerts or integrate with monitoring tools.

16.2 Common Commands for Monitoring

Command	Description
uptime	Show system uptime
top / htop	Real-time process monitoring
free -h	Check memory usage
df -h	Check disk usage
du -sh <dir>	Check directory size
ps aux	List running processes
pgrep <name>	Find process by name
netstat -tulnp	Check open ports
systemctl status <service>	Check service status

16.3 CPU Usage Monitoring Script

cpu_check.sh

```
#!/bin/bash
# Check CPU usage and alert if above threshold
THRESHOLD=80
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | \
           awk '{print $2}' | cut -d. -f1)
echo "CPU Usage: ${CPU_USAGE}%"
if [ $CPU_USAGE -gt $THRESHOLD ]; then
  echo "ALERT: CPU usage is high! (${CPU_USAGE}%)"
fi
```

Output (Normal)

```
$ bash cpu_check.sh
CPU Usage: 32%
```

Output (High Usage)

```
$ bash cpu_check.sh
CPU Usage: 92%
ALERT: CPU usage is high! (92%)
```

Key Points

- Uses top command to get CPU usage.
- Compares with threshold value.
- Can send email/Slack alerts.
- Useful for production monitoring.

16.4 Memory Usage Monitoring Script

memory_check.sh

```
#!/bin/bash
# Check memory usage
THRESHOLD=80
MEM_USAGE=$(free | awk '/Mem/ {print ($3/$2)*100}' | cut -d. -f1)
echo "Memory Usage: ${MEM_USAGE}%"
if [ $MEM_USAGE -gt $THRESHOLD ]; then
  echo "ALERT: Memory usage is high! (${MEM_USAGE}%)"
fi
```

Output (Normal)

```
$ bash memory_check.sh
Memory Usage: 45%
```

High Usage Output

```
$ bash memory_check.sh
Memory Usage: 88%
ALERT: Memory usage
is high! (88%)
```

16.5 Disk Usage Monitoring Script

disk_check.sh

```
#!/bin/bash
# Check disk usage for root partition
THRESHOLD=80
DISK_USAGE=$(df -h / | awk 'NR==2 {print $5}' | cut -d'%' -f1)
echo "Disk Usage: ${DISK_USAGE}%"
if [ $DISK_USAGE -gt $THRESHOLD ]; then
  echo "ALERT: Disk usage is high! (${DISK_USAGE}%)"
fi
```

Output (Normal)

```
$ bash disk_check.sh
Disk Usage: 42%
```

High Usage Output

```
$ bash disk_check.sh
Disk Usage: 95%
ALERT: Disk usage
is high! (95%)
```

16.6 Check Directory Size

dir_size.sh

```
#!/bin/bash
# Check size of a directory
DIR="/var/log"
echo "Directory: $DIR"
du -sh $DIR
```

Output

```
$ bash dir_size.sh
Directory: /var/log
1.2G    /var/log
```

16.7 Process Monitoring Script

process_check.sh

```
#!/bin/bash
# Check if a process is running
PROCESS="nginx"
if pgrep -x $PROCESS > /dev/null; then
  echo "$PROCESS is running."
else
  echo "$PROCESS is NOT running!"
fi
```

Output (Running)

```
$ bash process_check.sh
nginx is running.
```

Output (Not Running)

```
$ bash process_check.sh
nginx is NOT running!
```

16.8 Service Health Check Script

service_check.sh

```
#!/bin/bash
# Check service status
SERVICE="docker"
if systemctl is-active --quiet $SERVICE; then
  echo "$SERVICE service is running."
else
  echo "$SERVICE service is NOT running!"fi
```

Output (Running)

```
$ bash service_check.sh
docker service is running.
```

Output (Not Running)

```
$ bash service_check.sh
docker service is NOT running!
```

16.9 Send Email Alert (Using mail command)

email_alert.sh

```
#!/bin/bash
# Send email when disk usage is high
THRESHOLD=80
USAGE=$(df -h / | awk 'NR==2 {print $5}' | cut -d'%' -f1)
if [ $USAGE -gt $THRESHOLD ]; then
  echo "Disk usage is high: ${USAGE}%" | \
  mail -s "Disk Alert" rakesh@example.com
fi
```

16.10 Key Points

- Monitor CPU, memory, disk, processes and services.
- Use threshold values.
- Send email/Slack alerts on failure.
- Write logs for debugging.
- Useful in DevOps and production environments.

16.11 Real-World Use Cases

1. Monitor server health (CPU, Memory, Disk).
2. Automated log cleanup.
3. Database backup scripts.
4. Service health monitoring.
5. Send email alerts for failures.
6. Useful for DevOps automation and production environment.

17. Log Management & Backup Automation in Shell Scripting

17.1 Introduction

- Logs are important for troubleshooting, monitoring and auditing applications and servers.
- We can use shell scripts to manage logs, create backups, archive old files and clean up unnecessary data.
- Useful for DevOps, production support and system administration.
- Common tasks: log filtering, log rotation, archiving, backup automation and cleanup.

17.2 Common Log Management Commands

Command	Description
tail -f file	View live log
grep "pattern" file	Search for pattern in file
grep -i "error" file	Case-insensitive search
wc -l file	Count lines in file
head -n file	Show first N lines
tail -n file	Show last N lines
find /path -name "*.log"	Find log files
tar -czf file.tar.gz dir/	Create compressed archive
gzip file	Compress file
logrotate	Manage log rotation

17.3 View and Filter Logs

log_filter.sh

```
#!/bin/bash
# Filter ERROR logs from application log
LOG_FILE="/var/log/app.log"
echo "---ERROR logs ---"
grep -i "ERROR" $LOG_FILE

echo "---Last 10 lines ---"
tail -n 10 $LOG_FILE
```

Output

```
$ERROR 2026-10-20 12:23 ERROR
Database connection failed
2026-10-20 10:15:20 ERROR
Timeout while calling API
...
2026-10-20 10:16:21 INFO
Request processed successfully
2026-10-20 10:17:02 INFO
User login successful
```

Key Points

- ✓ Use grep, tail, head to analyze logs.
- ✓ Filter logs by date, error level or keyword.
- ✓ Helpful for troubleshooting application issues.
- ✓ Can be used in automation scripts to send alerts.

17.4 Create Log Archive (Backup)

log_backup.sh

```
#!/bin/bash
# Create backup of log directory
LOG_DIR="/var/log/app"
BACKUP_DIR="/backup/logs"
DATE= $(date +%Y-%m-%d_%H-%M-%S)
mkdir -p $BACKUP_DIR
tar -czf $BACKUP_DIR/app_logs_$DATE.tar.gz $LOG_DIR
echo "Backup created: app_logs_$DATE.tar.gz"
```

Output

```
$ bash log_backup.sh
Backup created:
app_logs_2026-10-20_10-30-15.tar.gz
```

Key Points

- ✓ Use tar with gzip for compression.
- ✓ Use date in filename for uniqueness.
- ✓ Useful for daily/weekly log backups.
- ✓ Store backups in different directory or remote server.

17.5 Log Rotation (Keep only last N days)

log_cleanup.sh

```
#!/bin/bash
# Delete log files older than 7 days
LOG_DIR="/var/log/app"
find $LOG_DIR -type f -name "*.log" -mtime +7 -delete
echo "Old log files deleted (older than 7 days)"
```

Output

```
$ bash log_cleanup.sh
Old log files deleted
(older than 7 days)
```

Key Points

- ✓ Find and remove old files.
- ✓ Helps in managing disk space.
- ✓ Can be scheduled with cron.
- ✓ Useful for production servers.

17.6 Backup with Retention (Keep last N backups)

backup_retention.sh

```
#!/bin/bash
# Keep only last 5 backups
# BACKUP_DIR="/backup/logs"
# N=5
ls -t1 $BACKUP_DIR | sort-r | tail -n + $((N+1)) | xargs -r rm -f
echo "Old backups deleted. Keeping last $N files."
```

Output

```
$ bash backup_retention.sh
Old backups deleted.
Keeping last 5 files.
$ ls /backup/logs
$ app_logs_2026-10-18.tar.gz
$ app_logs_2026-10-19.tar.gz
...
```

Key Points

- ✓ Keeps only last N backups.
- ✓ Uses ls -t1 to sort by time.
- ✓ Good for automated cleanup.
- ✓ Prevents disk space issues.

17.7 Copy Backup to Remote Server (SCP)

remote_backup.sh

```
#!/bin/bash
LOCAL_FILE="/backup/logs/app_logs_$(date +%F).tar.gz"
REMOTE_USER="ec2-user"
REMOTE_HOST="192.168.1.100"
REMOTE_DIR="/home/ec2-user/backups"
scp $LOCAL_FILE $REMOTE_USER@$REMOTE_HOST:$REMOTE_DIR
echo "Backup copied to remote server"
```

Output

```
$ bash remote_backup.sh
app_logs_2026-10-20.tar.gz
copied to 192.168.1.100
```

Key Points

- ✓ Use scp or rsync for backup.
- ✓ Requires SSH access and key.
- ✓ Useful for offsite backups.
- ✓ Can be automated with cron.

17.8 Important Points

- ✓ Use proper log level (ERROR, WARN, INFO, DEBUG).
- ✓ Rotate logs to avoid large file size.
- ✓ Keep backups with retention policy.
- ✓ Use cron for scheduling tasks.
- ✓ Ensure proper permissions and security.

17.9 Real-World Use Cases

1. Monitoring application logs in production.
2. Automated backup of logs and config files.
3. Sending alerts for critical errors.
4. Managing disk space on servers.
5. Useful in DevOps, cloud and system administration.

Manjunatha HG

18. Real-World DevOps Shell Scripting

18.1 Introduction

- Shell scripts are widely used in DevOps for automation, deployments, monitoring and maintenance.
- These real-world scripts help to save time, reduce manual work and improve reliability.
- Common use cases: application deployment, service management, Docker/Kubernetes operations, log rotation, backup, health checks and notifications.

18.2 List of Real-World DevOps Shell Scripts

1. Deployment automation script
2. Application health check script
3. Start/Stop/Restart service script
4. Docker container management script
5. Kubernetes pod management script
6. Database backup script
7. AWS resource status check script
8. Disk cleanup script
9. Send Slack/Email notification script

18.3 Application Deployment Script

deploy_app.sh

```bash
#!/bin/bash
# Simple deployment script
APP_NAME="myapp"
APP_DIR="/opt/$APP_NAME"
echo "Stopping old application..."
sudo systemctl stop $APP_NAME 2>/dev/null
cd $APP_DIR || { echo "Directory not found"; exit 1; }
git pull origin main
mvn clean package -DskipTests
sudo systemctl start $APP_NAME
echo "Deployment completed successfully!"
```

Output

```
$ bash deploy_app.sh
Stopping old application...
Already up to date.
[INFO] BUILD SUCCESS
Starting application...
Deployment completed
successfully!
```

Key Points

- ✔ Automates code pull, build and deployment.
- ✔ Restarts application service.
- ✔ Useful in CI/CD pipelines.
- ✔ Can be integrated with Jenkins, GitHub Actions or cron jobs.

18.4 Application Health Check Script

health_check.sh

```bash
#!/bin/bash
# Check if application is running
URL="http://localhost:8080/health"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" $URL)
if [ $HTTP_CODE -eq 200 ]; then
    echo "Application is healthy (HTTP 200)"
else
    echo "Application is down! (HTTP $HTTP_CODE)"
    # Send notification (optional)
fi
```

Output (Healthy)

```
$ bash health_check.sh
Application is healthy
(HTTP 200)
```

Output (Down)

```
$ bash health_check.sh
Application is down!
(HTTP 503)
```

Key Points

- ✔ Checks application health endpoint.
- ✔ Useful for monitoring and alerting.
- ✔ Can be scheduled using cron.
- ✔ Can send notifications on failure.

18.5 Service Management Script

manage_service.sh

```bash
#!/bin/bash
# Start | Stop | Restart | Status a service
SERVICE_NAME="nginx"
ACTION=$1
case $ACTION in
    start)   systemctl start $SERVICE_NAME ;;
    stop)    systemctl stop $SERVICE_NAME ;;
    restart) systemctl restart $SERVICE_NAME ;;
    status)  systemctl status $SERVICE_NAME ;;
    *) echo "Usage: $0 {start|stop|restart|status}" ;;
esac
```

Output

```
$ bash manage_service.sh status
nginx.service - A high
performance web server
  Active: active (running)
  Loaded: loaded (/usr/lib/...)
```

Key Points

- ✔ Manages system services using systemctl.
- ✔ Supports start, stop, restart and status.
- ✔ Can be used for multiple services.
- ✔ Useful for production maintenance.

18.6 Docker Container Management Script

docker_manage.sh

```bash
#!/bin/bash
# Manage docker containers
ACTION=$1
CONTAINER_NAME="myapp"
case $ACTION in
    start)   docker start $CONTAINER_NAME ;;
    stop)    docker stop $CONTAINER_NAME ;;
    restart) docker restart $CONTAINER_NAME ;;
    logs)    docker logs -f $CONTAINER_NAME ;;
    *) echo "Usage: $0 {start|stop|restart|status}" ;;
esac
```

Output

```
$ bash docker_manage.sh start
myapp
$ bash docker_manage.sh logs
Attaching to myapp...
myapp  | Server started
myapp  | Listening on port 8080
```

Key Points

- ✔ Manage docker containers easily.
- ✔ Useful for troubleshooting.
- ✔ Can be extended to handle docker compose.
- ✔ Useful in CI/CD and production environments.

18.7 Kubernetes Pod Management Script

k8s_pod.sh

```bash
#!/bin/bash
# Check pods and restart if not running
NAMESPACE="default"
POD_NAME=$1
STATUS=$(kubectl get pod $POD_NAME -n $NAMESPACE \
  -o jsonpath='{.status.phase}')
if [ "$STATUS" != "Running" ]; then
    echo "Pod $POD_NAME is not running. Restarting..."
    kubectl delete pod $POD_NAME -n $NAMESPACE
else
    echo "Pod $POD_NAME is running."
fi
```

Output

```
$ bash k8s_pod.sh myapp-pod
Pod myapp-pod is not running.
Restarting...
pod "myapp-pod" deleted
Pod myapp-pod is running.
```

Key Points

- ✔ Checks Kubernetes pod status.
- ✔ Automatically restarts failed pods.
- ✔ Useful for self-healing mechanisms.
- ✔ Can be extended for multiple pods or namespaces.

18.8 Important Points

- ✔ Use proper log level (ERROR, WARN, INFO, DEBUG).
- ✔ Rotate logs to avoid large file size.
- ✔ Keep backups with retention policy.
- ✔ Use cron for scheduling tasks.
- ✔ Add email/Slack notifications for important events.
- ✔ Ensure proper permissions and security.

18.9 Real-World Use Cases

1. Monitor application logs in production.
2. Automated backup of logs and configuration files.
3. Service health monitoring and auto-restart.
4. Docker/Kubernetes operations and maintenance.
5. Sending alerts for failures (email/Slack).
6. Useful in DevOps, cloud and system administration.